

Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab Report 03: Processes and Threads

Student Name	Muhammad Fahad Noor
Registration No.	232448
Section	BSCS-ev-7-C
Instructor	Muhammad Nauman Zafar Hashmi
Date	Wednesday, September 23, 2026

Section 1: Environment Setup

Confirmed via `dotnet --version` that the .NET SDK carried over from Lab 02 is still available. A working folder `PDC_Lab03` was created, containing four sub-projects: `ProcessLab`, `ArraySumThreads`, `CreationOverhead`, and `ThreadLifecycle`.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC>dotnet --version
10.0.401
```

Section 2: Task 1 — Process Creation and Address-Space Separation

Source file: `ProcessLab/Program.cs`

```
using System;
using System.Diagnostics;

class Program
{
    static void Main(string[] args)
    {
        if (args.Length > 0 && args[0] == "--child")
        {
            RunAsChild();
        }
        else
        {
            RunAsParent();
        }
    }

    static void RunAsChild()
    {
        Console.WriteLine($"[Child] PID = {Environment.ProcessId}");
        int counter = 100;
        counter += 50;
        Console.WriteLine($"[Child] final counter = {counter}");
    }

    static void RunAsParent()
    {
        Console.WriteLine($"[Parent] PID = {Environment.ProcessId}");
        int counter = 100;
        counter += 1;

        string exePath = Environment.ProcessPath!;

        var psi = new ProcessStartInfo
        {
            FileName = exePath,
            UseShellExecute = false
        };
        psi.ArgumentList.Add("--child");

        Process child = Process.Start(psi)!;
        child.WaitForExit();

        Console.WriteLine($"[Parent] final counter = {counter}");
        Console.WriteLine("[Parent] Parent and child counters were modified independently (separate address spaces).");
    }
}
```

Results

Role	Process ID (PID)	Final Counter Value
Parent	1208	101
Child	20300	150

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC Repo\Lab3 PDC\ProcessLab>dotnet run
[Parent] PID = 1208
[Child] PID = 20300
[Child] final counter = 150
[Parent] final counter = 101
[Parent] Parent and child counters were modified independently (separate address spaces).
```

Discussion Questions

1. Were the parent's and child's process IDs different? What does this confirm about how Process.Start() creates the child?

Yes. The parent ran as PID 1208 and the child as PID 20300. This confirms that Process.Start() asks the operating system to create a completely new, independent process with its own PID, its own address space and its own copy of the .NET runtime, loaded fresh from the executable. The child is not a thread inside the parent. It is a separate program instance that the OS schedules and manages on its own.

2. Both the parent and the child set a local counter to 100 and then modified it differently. Did either process's final counter value affect the other's? Explain why, in terms of address-space separation.

No. The parent finished with 101 and the child with 150, and neither value affected the other. Each process has its own private virtual address space, so the variable named counter in the parent and the one in the child sit in different physical memory, even though they have the same name and the same starting value. The OS keeps these address spaces isolated, so one process cannot see or change another's memory without explicit inter-process communication.

3. On Linux, fork() duplicates the parent's entire memory image at the moment of the call, so the child briefly starts with an exact copy of the parent's variables. In this C# program using Process.Start(), does the child ever see the parent's in-progress counter value (i.e. 101, after the parent's increment) at any point? Explain the difference between fork()'s copy-on-write duplication and launching a fresh process with Process.Start().

No. The child never sees 101. Process.Start() launches a brand-new program image from the .exe. The child starts from Main(), goes into RunAsChild() and creates its own counter = 100 from scratch, and nothing from the parent's memory is inherited except the command-line arguments and environment. On Linux, fork() instead duplicates the parent's memory using copy-on-write: both processes share the same pages read-only until one of them writes, and only then is that page copied. So a forked child would start out seeing counter = 101. Process.Start() behaves like fork() + exec() (or Windows CreateProcess), which replaces the memory image completely.

Section 3: Task 2 — Summing Array Slices Across Worker Threads

Source file: ArraySumThreads/Program.cs

```
using System;
using System.Threading;

class Program
{
    static long[] data = new long[10_000_000];
    static long[] partialSums = Array.Empty<long>();
    static int numWorkers;

    static void SumSlice(object? arg)
    {
        int idx = (int)arg!;
        int sliceSize = data.Length / numWorkers;
        int start = idx * sliceSize;
        int end = (idx == numWorkers - 1) ? data.Length : start + sliceSize;

        long sum = 0;
        for (int i = start; i < end; i++)
        {
            sum += data[i];
        }
        partialSums[idx] = sum;
    }

    static void Main()
    {
        for (int i = 0; i < data.Length; i++) data[i] = i + 1;

        numWorkers = Environment.ProcessorCount;
        partialSums = new long[numWorkers];

        Thread[] threads = new Thread[numWorkers];

        for (int i = 0; i < numWorkers; i++)
        {
            int idx = i;
            threads[i] = new Thread(() => SumSlice(idx));
            threads[i].Start();
        }

        for (int i = 0; i < numWorkers; i++)
        {
            threads[i].Join();
        }

        long threadedTotal = 0;
        foreach (long partial in partialSums) threadedTotal += partial;

        long sequentialTotal = 0;
        foreach (long value in data) sequentialTotal += value;

        Console.WriteLine($"Array size: {data.Length}");
        Console.WriteLine($"Worker threads used: {numWorkers}");
        Console.WriteLine($"Threaded total: {threadedTotal}");
        Console.WriteLine($"Sequential total: {sequentialTotal}");
        Console.WriteLine($"Match: {threadedTotal == sequentialTotal}");
    }
}
```

Results

Array Size	Worker Threads Used	Threaded Total	Sequential Total	Match? (Y/N)
10,000,000	8	50,000,005,000,000	50,000,005,000,000	Y

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab3_PDC\ArraySumThreads>dotnet run
Array size: 10000000
Worker threads used: 8
Threaded total: 50000005000000
Sequential total: 50000005000000
Match: True
```

Discussion Questions

1. Why does each worker thread writing only to its own slot in partialSums avoid the need for a lock, whereas having every thread add directly into one shared long total variable would not?

Each worker writes only to partialSums[idx], its own separate array slot, so no two threads ever write to the same memory location and there is no data race. The main thread reads the slots only after Join(), which guarantees every worker has finished and its write is visible. If all threads did total += sum on one shared variable, that statement would be a non-atomic read-modify-write: two threads could read the same old value, and one update would overwrite the other and be lost. That would need a lock or Interlocked.Add() to be correct.

2. If the array length is not evenly divisible by the number of worker threads, how does the skeleton above avoid dropping the last few elements? Identify the specific line responsible.

The responsible line is: `int end = (idx == numWorkers - 1) ? data.Length : start + sliceSize;` The slice size uses integer division, which can leave a few leftover elements. The last worker does not stop at `start + sliceSize`. It runs all the way to `data.Length`, so it picks up any remainder. In our run, $10,000,000 / 8 = 1,250,000$ divides exactly, but with 6 threads, for example, the last worker would take the 4 extra elements.

3. How does this task relate to Task 1: which parts of the ArraySumThreads program are shared between all the worker threads (as would be true within a single process), and how does that sharing differ from the address-space separation you observed between the parent and child processes in Task 1?

All worker threads run inside one process and share one address space. The static data array, the partialSums array and numWorkers are visible to every thread directly, with no copying. Only each thread's local variables (sum, i, start, end) are private, because each thread has its own stack. This is the opposite of Task 1: there the parent and child had fully separate address spaces, so a change in one never reached the other. Threads can share data cheaply, but that is exactly why they must avoid writing to the same location at the same time.

Section 4: Task 3 — Measuring Process- vs. Thread-Creation Overhead

Source file: CreationOverhead/Program.cs

```
using System;
using System.Diagnostics;
using System.Threading;

class Program
{
    const int Iterations = 50;

    static readonly string ChildExePath =
        @"..\ProcessLab\bin\Release\net10.0\ProcessLab.exe";

    static void Main()
    {
        if (!System.IO.File.Exists(ChildExePath))
        {
            Console.WriteLine($"ERROR: Could not find {ChildExePath}");
            Console.WriteLine("Build ProcessLab in Release mode first, then fix the ChildExePath above.");
            return;
        }

        var processStopwatch = Stopwatch.StartNew();
        for (int i = 0; i < Iterations; i++)
        {
            var psi = new ProcessStartInfo
            {
                FileName = ChildExePath,
                UseShellExecute = false
            };
            psi.ArgumentList.Add("--child");

            Process p = Process.Start(psi);
            p.WaitForExit();
        }
        processStopwatch.Stop();

        var threadStopwatch = Stopwatch.StartNew();
        for (int i = 0; i < Iterations; i++)
        {
            Thread t = new Thread(() => { /* trivial work */ });
            t.Start();
            t.Join();
        }
        threadStopwatch.Stop();

        double avgProcessMs = processStopwatch.Elapsed.TotalMilliseconds / Iterations;
        double avgThreadMs = threadStopwatch.Elapsed.TotalMilliseconds / Iterations;

        Console.WriteLine($"Average process creation time: {avgProcessMs:F3} ms");
        Console.WriteLine($"Average thread creation time: {avgThreadMs:F3} ms");
        Console.WriteLine($"Process creation was {(avgProcessMs / avgThreadMs):F1}x more expensive than thread creation.");
    }
}
```

Results

Run	Avg. Process Creation (ms)	Avg. Thread Creation (ms)	Ratio (Process / Thread)
1	69.596	0.147	473.2x
2	65.041	0.144	452.6x

```

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab3_PDC\CreationOverhead>dotnet run
[Child] PID = 18928
[Child] final counter = 150
[Child] PID = 22120
[Child] final counter = 150
[Child] PID = 17208
[Child] final counter = 150
[Child] PID = 11260
[Child] final counter = 150
[Child] PID = 20496
[Child] final counter = 150

```

```

[Child] final counter = 150
[Child] PID = 21352
[Child] final counter = 150
Average process creation time: 69.596 ms
Average thread creation time: 0.147 ms
Process creation was 473.2x more expensive than thread creation.

```

Discussion Questions

1. Which was more expensive to create, a process or a thread, and by roughly what factor on your machine?

Process creation was far more expensive. Across two runs, creating a process averaged 69.596 ms and 65.041 ms, while creating a thread averaged only 0.147 ms and 0.144 ms. That makes process creation roughly 450–475 times more expensive than thread creation on my machine, and the ratio was consistent between runs.

2. Referencing the Background section, explain in your own words why creating a new process (a new address space and OS resource set) costs more than creating a new thread (which reuses the existing process's address space).

To create a new process, the OS must set up a whole new virtual address space and page tables, create a process control block and handle table, load the executable and its DLLs from disk, and start and initialise a new .NET runtime (JIT, garbage collector, etc.) before the first line of Main() runs. All of this is torn down again when the process exits. A new thread reuses everything the process already has (code, heap, loaded runtime, open resources), so the OS only needs to allocate a stack and a thread control block and add it to the scheduler.

3. Would you expect this ratio to grow, shrink, or stay roughly the same if each trivial unit of work were replaced with something more substantial (e.g. summing a large array, as in Task 2)? Justify your answer.

I would expect the ratio to shrink toward 1. The creation cost is a fixed overhead paid once per unit of work, while the useful work takes the same time whether it runs in a process or a thread. As the work gets larger, it dominates the total time, and the creation overhead becomes a small fraction of it. For example, if each unit did 1000 ms of work, the comparison would be about 1069.6 ms vs 1000.1 ms, a ratio of only about 1.07x instead of 473x.

Section 5: Task 4 - Thread Lifecycle States

Source file: ThreadLifecycle/Program.cs

```
using System;
using System.Threading;

class Program
{
    static void Worker()
    {
        Thread.Sleep(200);
    }

    static void Main()
    {
        Thread t = new Thread(Worker);

        Console.WriteLine($"After creation: {t.ThreadState}"); // New

        t.Start();

        Console.WriteLine($"Immediately after Start(): {t.ThreadState}"); // Runnable/Ready or Running

        Thread.Sleep(50);

        Console.WriteLine($"While worker is sleeping: {t.ThreadState}"); // Blocked/Waiting

        t.Join();

        Console.WriteLine($"After Join() completes: {t.ThreadState}"); // Terminated
    }
}
```

Results

Point in Program	Thread.ThreadState Reported	Conceptual Lifecycle State
Immediately after creation, before Start()	Unstarted	New
Immediately after Start()	Running	Runnable / Ready (or Running)
While the worker is inside Thread.Sleep(200)	WaitSleepJoin	Blocked / Waiting
After Join() has returned	Stopped	Terminated

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC Repo\Lab3_PDC\ThreadLifecycle>dotnet run
After creation: Unstarted
Immediately after Start(): Running
While worker is sleeping: WaitSleepJoin
After Join() completes: Stopped
```

Discussion Questions

1. Using your Task 3 measurements and the lifecycle states above, explain why moving a thread through New -> Runnable -> Running is cheaper than the equivalent process-creation path observed in Task 3.

Moving a thread from New (Unstarted) through Runnable to Running is only scheduler bookkeeping inside an existing process. Start() allocates a stack and puts the thread in the ready queue, and it is reported as Running almost immediately. The whole create-start-join cycle took only 0.147 ms in

Task 3. A process has to go through far more work before it can reach its running state: a new address space, loading the executable and starting a new runtime. That is why its path averaged 69.596 ms. (Note: .NET reports both Ready and Running as Running.)

2. A program that needs to perform thousands of small, short-lived units of work (fine-grained parallelism) is generally implemented with threads rather than processes. Using your results from Tasks 2, 3, and this task, construct an argument for why: consider creation overhead, address-space sharing (from Task 2), and how quickly a unit of work can move through the lifecycle states.

Threads are the better choice for fine-grained work for three reasons. First, creation overhead: at 0.147 ms per thread vs 69.6 ms per process (Task 3), 1000 small tasks would spend about 0.15 s on thread creation but about 70 s on process creation, which is more than the work itself. Second, address-space sharing: as in Task 2, threads read the same input array and write results into a shared array directly, with no copying or inter-process communication. Third, lifecycle speed: this task showed a thread moving from Unstarted to Running to WaitSleepJoin to Stopped within a few hundred milliseconds, mostly the sleep itself, so short units of work move through their lifecycle quickly. Threads can also be reused through a thread pool, which removes even the creation cost.

3. Give one scenario where the address-space isolation demonstrated in Task 1 (processes, not threads) would be preferred despite its higher creation overhead. Justify your answer.

A web browser running each tab or plugin in its own process (as Chrome does) is a good example. Web pages run untrusted code and can crash. With process isolation, a crash or malicious script in one tab cannot read or corrupt the memory of other tabs or the main browser, because the OS enforces separate address spaces (exactly what Task 1 showed). Tabs are long-lived, so paying about 70 ms once when a tab opens is a small price for the security and fault tolerance.

Section 6: GitHub Repository

The source code for all four Lab 03 projects (ProcessLab, ArraySumThreads, CreationOverhead, ThreadLifecycle) is on GitHub. Each lab is kept on its own branch; Lab 03 is on the lab3 branch.

- **Repository:** https://github.com/fahad1084/-PDC_Labs
- **Lab 03 branch:** https://github.com/fahad1084/-PDC_Labs/tree/lab3